## 8. CORE MATHEMATICAL FORMULATIONS & SAFE MATH ENGINE

To eliminate AI hallucinations, all mathematical operations are implemented in pure Python (src/metrics.py).
The core safe_divide(numerator, denominator) utility guarantees that zero denominators, negative bases, None,
and NaN values never trigger uncaught runtime exceptions, ensuring strict mathematical consistency across the system.

| Metric Name | Mathematical Formula | Zero / Edge Case Condition | Engine Behavior & Output |
|---|---|---|---|
| Target Achievement | (Revenue / Target) * 100 | Target Quota = 0 | safe_divide returns None. Renders 'Not Available'. |
| Sales Growth (MoM) | ((Current - Prior) / Prior) * 100 | Prior Period Sales = 0 | Growth is mathematically undefined; safe_divide returns None. |
| First-Month Growth | ((Current - Prior) / Prior) * 100 | First month in dataset (2026-08) | Prior is None. Flags is_first_month=True; displays 'N/A'. |
| Gross Margin ($) | Revenue - (Units * Unit Cost) | Unit Cost missing in margins.xlsx | Returns status='MISSING_COST'; margin is None (Flagged). |
| Gross Margin Rate | (Gross Margin / Revenue) * 100 | Revenue = 0 or Cost Missing | safe_divide prevents division by zero; returns None. |
| Return Rate (%) | (Returns / Units) * 100 | Units Sold = 0 | safe_divide returns None; prevents ZeroDivisionError. |
| Return Anomaly | Returns > Units Sold | returns = 350, units = 320 | Raises ValueError('Returns cannot exceed units sold'). |
| Standard Discount | Playbook Regex Extraction | Discount requested <= 10% | validate_discount_policy returns status='APPROVED'. |
| Exception Discount | Playbook Regex Extraction | Discount > 10% with Director sign-off | Returns status='APPROVED_WITH_EXCEPTION'. |
| Prohibited Discount | Playbook Regex Extraction | Discount > 10% without Director approval | Returns status='BLOCKED'; strictly declines unapproved quote. |

## 9. CONDITIONAL ROUTING BRANCHES & DIFFERENTIATED REMEDIATION

The Advisory Agent implements strict non-conflation rules to separate underlying commercial issues:

**Branch A: Low Revenue** — Trigger: Achievement < 90% or MoM Growth < 0%

Prescribed Action: Qualify deal pipeline velocity, launch co-op push campaigns with partners, deploy non-cash support bundles. NEVER issue panic price cuts.

**Branch B: Low Margin** — Trigger: Gross Margin < 15.0% or Landed Cost Surge

Prescribed Action: Immediately halt discretionary discounting. Attach high-margin accessories and software licenses. Renegotiate distributor landed costs.

**Branch C: High Returns** — Trigger: Return Rate > 5.0% of Shipped Units

Prescribed Action: Quarantine defective hardware revisions, initiate QA manufacturing inspection, provide customer integration training. STRICTLY PROHIBIT DISCOUNTS.

## 10. RESPONSIBLE AI & CAUSAL GOVERNANCE

- Correlation vs Causation: The agent explicitly treats data patterns as correlations, avoiding unfounded assertions.
- Dynamic Policy Ingestion: Discount limits are extracted via regular expressions directly from the playbook (never hardcoded).
- Bounded RAG Grading: Query rewriting is restricted to a maximum of 2 retries, ensuring bounded execution.
- Idempotent State Storage: Duplicate saves for the same plan parameters return the existing ID, preventing ledger bloating.

## 11. SAFE DIVISION IMPLEMENTATION (PYTHON SNIPPET)

```python
def safe_divide(numerator: Optional[Union[int, float]], denominator: Optional[Union[int, float]]) -> Optional[float]:
    if numerator is None or denominator is None:
        return None
    try: num, den = float(numerator), float(denominator)
    except (ValueError, TypeError): return None
    if math.isnan(num) or math.isnan(den) or den <= 0: return None
    if num == 0.0: return 0.0
    return num / den
```